

Module 7 Week A — Stretch: Adversarial Evaluation

Honors Track. This stretch is for learners who have completed all core assignments, are On Track or Advanced in the program, and are attending consistently.

Prerequisite: Integration Task 7A base must be substantially complete — your fine-tuned model is on Hugging Face Hub and you have a domain-shift analysis from the integration task. Both inform the hypotheses you form here.

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

Objective

Construct a small, hand-crafted *adversarial* test set (~30 examples) targeting hypothesized failure modes of your fine-tuned classifier, then run the classifier against this set and write a 1-page memo on what the results reveal about the model's decision boundary.

The pedagogical goal is to move from "evaluate on a held-out test set" to "design tests that probe specific weaknesses." Constructing adversarial cases requires you to *form hypotheses* about model behavior, not just measure it.

Repo and Branch

This stretch **extends your Lab 7A repo**. The Tuesday stretch (if you did it) is on a separate branch — this one is independent. Create a new branch:

```
cd <your-lab-7a-repo>
git checkout main
git checkout -b stretch-thu-adversarial
```

Your local `model/` from Lab 7A is what you'll use for the actual evaluation. The starter assets live in `stretch/thursday/` of the Lab 7A template repo:

```
cp -r stretch/thursday/* .
```

What You're Building

Files

- `adversarial_set.csv` — your hand-crafted adversarial dataset, ~30 rows beyond the 3 starter rows
- `run_adversarial.py` — runs the model against your adversarial set, writes results
- `adversarial-analysis.md` — 1-page memo on findings
- `results.csv` — generated output (model predictions on the adversarial set)

Hypothesis categories

Each adversarial example targets one of these hypothesized failure modes:

- negation** — sentences whose sentiment depends on a negation (e.g., "did not improve", "fails to grow"). Hypothesis: the model misses the negation cue.
- lexical_trigger** — reviews containing a strong sentiment word in a context that flips its polarity (e.g., "no longer reliable" — `reliable` is a positive trigger but the review is negative).
- domain_shift** — sentences from a different domain (news prose, sports, recipes) where app-review-trained cues don't apply.
- length_extreme** — very short (3–5 words) or very long (50+ words) sentences.
- sarcasm** — sentences whose surface form is one polarity but whose intended meaning is the opposite.
- other** — your own category; explain in the rationale.

`adversarial_set.csv` schema

Column	Type	Description
<code>id</code>	int	Unique row id
<code>hypothesis_category</code>	enum	One of the categories above
<code>text</code>	str	The adversarial example
<code>expected_label</code>	str	What a human would label (<code>positive</code> / <code>neutral</code> / <code>negative</code>)
<code>rationale</code>	str	1 sentence explaining why this example targets this hypothesis

Distribution requirement: Each non-`other` category must have at least 3 examples. Total ~30 rows.

The `rationale` is what the TA reads to determine if your set is genuinely hypothesis-driven (and not random or AI-generated). Be specific: "This example uses the negation of a strong positive cue ('did not improve') to test whether the model picks up the negation or just the cue word."

```
run_adversarial.py

def load_model(model_path: str = "model"):
    """Load model and tokenizer from a local path. Default: 'model'."""

def run_against_set(adv_csv_path: str, model, tokenizer) -> pd.DataFrame:
    """
    Run the model on every row of adv_csv_path; return a DataFrame with all
    original columns plus predicted_label, predicted_probability, correct.

    Label names must be read from model.config.id2label — do not hard-code.
    """

def main() -> None:
    """Orchestrate; write results.csv."""
```

The CI uses a substitute public sentiment model (`distilbert-base-uncased-finetuned-sst-2-english`) so the pipeline runs without your local checkpoint. Reading labels from `model.config.id2label` is required: the substitute is binary, your real model is 3-class — same code must work against both.

Writing `adversarial-analysis.md`

Required sections:

```
## Per-hypothesis accuracy
A table: for each hypothesis category, count of correct vs. incorrect predictions on your adversarial set. Mention the overall accuracy.

## Confirmed hypotheses
Which categories did the model fail on as you predicted? Cite specific row IDs and predictions (don't speak generally).

## Refuted hypotheses
Which categories did the model handle better than you expected? Be honest — this is the harder section to write but the more interesting one.

## What the results reveal about the decision boundary
Articulate one or more specific things the adversarial results say about how the model decides. ("The model is over-reliant on cue words like 'profit' regardless of negation" is the right specificity level.)
```

The "refuted" and "decision boundary" sections are what TAs read most carefully — they're what distinguish a hypothesis-driven adversarial study from "I generated 30 examples and ran them."

Submission

Open a PR from `stretch-thu-adversarial` into your Lab 7A `main`. The PR is the deliverable.

PR description includes:

- Per-category accuracy summary (1 line per category)
- One specific decision-boundary insight (1 sentence)

Submission instruction: Paste your PR URL into TalentLMS → Module 7 → Stretch — Thursday (Honors) to submit this assignment.

A note on AI generation

Adversarial datasets are easy to generate with an LLM. They will look polished and have plausible rationales. They will also lack the hypothesis-driven design that makes the exercise valuable.

If you use an LLM to generate examples, your rationales will read generically and your decision-boundary memo will be vague. Reviewers screen for this. Your time is better spent constructing 30 examples by hand — the *act* of designing them is the learning.